

✓ P1 Airline Project — Data Quality Checks

Phase 1 — Final Task

Verify row counts, nulls, duplicates, outliers, date coverage, and referential integrity across all 4 tables.

```
# Cell 1 — Install & Connect
!pip install psycopg2-binary sqlalchemy -q

from sqlalchemy import create_engine, text
import pandas as pd
import getpass

DB_PASSWORD = getpass.getpass('Enter Neon password: ')

conn_string = (
    f"postgresql+psycopg2://neondb_owner:{DB_PASSWORD}"
    f"@ep-super-cell-aofus8nt-pooler.c-2.ap-southeast-1.aws.neon.tech"
    f"/neondb?sslmode=require"
)
engine = create_engine(conn_string)

with engine.connect() as conn:
    version = conn.execute(text('SELECT version()')).scalar()
    print('Connected:', version)
```

0.0/4.3 MB ? eta -:-:-npg_8GWDIsBC5PkM

4.3/4.3 MB 11.8 MB/s eta 0:00:00

Enter Neon password:

Connected: PostgreSQL 18.4 (72c6e7c) on aarch64-unknown-linux-gnu, compiled by gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0, 64-bit

```
# Cell 2 — Row Counts
with engine.connect() as conn:
    total_flights = conn.execute(text('SELECT COUNT(*) FROM flights')).scalar()
    total_delays = conn.execute(text('SELECT COUNT(*) FROM delays')).scalar()
    total_airports = conn.execute(text('SELECT COUNT(*) FROM airports')).scalar()
    total_carriers = conn.execute(text('SELECT COUNT(*) FROM carriers')).scalar()

print('=' * 50)
print(' ROW COUNTS')
print('=' * 50)
print(f' Flights table  : {total_flights:>10,}')
print(f' Delays table   : {total_delays:>10,}')
print(f' Airports table  : {total_airports:>10,}')
print(f' Carriers table  : {total_carriers:>10,}')
print()
if total_flights == total_delays:
    print('OK Flights and Delays row counts match')
else:
    print(f'MISMATCH — Flights: {total_flights:,} | Delays: {total_delays:,}')
```

```
=====
ROW COUNTS
=====
Flights table  : 480,000
Delays table   : 480,000
Airports table : 351
Carriers table : 14
```

OK Flights and Delays row counts match

```
# Cell 3 — Null Checks
null_queries = {
    'null_carrier_code' : 'SELECT COUNT(*) FROM flights WHERE carrier_code IS NULL',
    'null_origin'       : 'SELECT COUNT(*) FROM flights WHERE origin IS NULL',
    'null_dest'         : 'SELECT COUNT(*) FROM flights WHERE dest IS NULL',
    'null_flight_date'  : 'SELECT COUNT(*) FROM flights WHERE flight_date IS NULL',
    'null_arr_delay'    : 'SELECT COUNT(*) FROM flights WHERE arr_delay_min IS NULL',
    'null_dep_delay'    : 'SELECT COUNT(*) FROM flights WHERE dep_delay_min IS NULL',
```

```

'null_distance' : 'SELECT COUNT(*) FROM flights WHERE distance IS NULL',
'null_air_time' : 'SELECT COUNT(*) FROM flights WHERE air_time IS NULL',
}

print('=' * 50)
print(' NULL CHECKS (should all be 0)')
print('=' * 50)

all_clean = True
with engine.connect() as conn:
    for label, query in null_queries.items():
        count = conn.execute(text(query)).scalar()
        flag = 'OK ' if count == 0 else 'XX '
        if count > 0:
            all_clean = False
        print(f' {flag} {label:<25} : {count:>8,}')

print()
print('No nuls found in critical columns' if all_clean else 'Nulls found — review above')

```

```

=====
NULL CHECKS (should all be 0)
=====
OK null_carrier_code      :    0
OK null_origin            :    0
OK null_dest              :    0
OK null_flight_date       :    0
OK null_arr_delay         :    0
OK null_dep_delay         :    0
OK null_distance          :    0
XX null_air_time          : 8,441

```

Nulls found — review above

Cell 4 — Duplicate Check

```

dup_query = """
SELECT COUNT(*) FROM (
    SELECT carrier_code, flight_number, flight_date, origin, dest,
           COUNT(*) AS cnt
    FROM flights
    GROUP BY carrier_code, flight_number, flight_date, origin, dest
    HAVING COUNT(*) > 1
) t
"""

```

```

with engine.connect() as conn:
    duplicates = conn.execute(text(dup_query)).scalar()

print('=' * 50)
print(' DUPLICATE CHECK')
print('=' * 50)
if duplicates == 0:
    print(' OK No duplicate flights found')
else:
    print(f' XX {duplicates:,} duplicate flight combinations found')

```

```

=====
DUPLICATE CHECK
=====
OK No duplicate flights found

```

Cell 5 — Outlier Detection

```

outlier_queries = {
    'Delays > 500 min (extreme)' : 'SELECT COUNT(*) FROM flights WHERE arr_delay_min > 500',
    'Delays < -120 min (too early)' : 'SELECT COUNT(*) FROM flights WHERE arr_delay_min < -120',
    'Distance = 0 or NULL' : 'SELECT COUNT(*) FROM flights WHERE distance <= 0 OR distance IS NULL',
    'Air time = 0 or NULL' : 'SELECT COUNT(*) FROM flights WHERE air_time <= 0 OR air_time IS NULL',
    'Cancelled but delay > 0' : 'SELECT COUNT(*) FROM flights WHERE cancelled = 1 AND arr_delay_min > 0',
}

print('=' * 50)
print(' OUTLIER DETECTION')
print('=' * 50)

```

```
with engine.connect() as conn:
    for label, query in outlier_queries.items():
        count = conn.execute(text(query)).scalar()
        flag = 'OK ' if count == 0 else 'WW '
        print(f' {flag} {label:<35} : {count:>8,}')

    max_delay = conn.execute(text('SELECT MAX(arr_delay_min) FROM flights')).scalar()
    min_delay = conn.execute(text('SELECT MIN(arr_delay_min) FROM flights')).scalar()
    avg_delay = conn.execute(text(
        'SELECT ROUND(AVG(arr_delay_min)::numeric, 2) FROM flights WHERE cancelled = 0'
    )).scalar()

print()
print(f' Max arrival delay : {max_delay} min')
print(f' Min arrival delay : {min_delay} min')
print(f' Avg arrival delay : {avg_delay} min (non-cancelled flights only)')
```

```
=====
OUTLIER DETECTION
=====
WW Delays > 500 min (extreme)      :   889
OK Delays < -120 min (too early)   :     0
OK Distance = 0 or NULL           :     0
WW Air time = 0 or NULL           :  8,441
OK Cancelled but delay > 0        :     0

Max arrival delay : 3275.0 min
Min arrival delay : 0.0 min
Avg arrival delay : 17.06 min (non-cancelled flights only)
```

```
# Cell 6 — Date Range and Monthly Coverage
with engine.connect() as conn:
    date_range = conn.execute(text(
        "SELECT MIN(flight_date)::text || ' to ' || MAX(flight_date)::text FROM flights"
    )).scalar()

    monthly = pd.read_sql(text("""
        SELECT
            TO_CHAR(flight_date, 'YYYY-MM') AS month,
            COUNT(*) AS total_flights,
            SUM(cancelled) AS cancelled_flights
        FROM flights
        GROUP BY TO_CHAR(flight_date, 'YYYY-MM')
        ORDER BY month
        """), conn)

print('=' * 50)
print(' DATE RANGE AND MONTHLY COVERAGE')
print('=' * 50)
print(f' Full range : {date_range}')
print(f' Months    : {len(monthly)}')
print()
print(monthly.to_string(index=False))
print()
if len(monthly) == 12:
    print(' OK All 12 months present')
else:
    print(f' WW Only {len(monthly)} months found — expected 12')
```

```
=====
DATE RANGE AND MONTHLY COVERAGE
=====
Full range : 2025-01-01 to 2025-12-31
Months    : 12

month total_flights cancelled_flights
2025-01      40000         1188
2025-02      40000          562
2025-03      40000          470
2025-04      40000          330
2025-05      40000          463
2025-06      40000          584
2025-07      40000         1024
2025-08      40000          425
```

2025-09	40000	200
2025-10	40000	220
2025-11	40000	1065
2025-12	40000	593

OK All 12 months present

```
# Cell 7 — Referential Integrity
integrity_queries = {
    'Flights missing delay record' : """
        SELECT COUNT(*) FROM flights f
        LEFT JOIN delays d ON f.flight_id = d.flight_id
        WHERE d.flight_id IS NULL
    """,
    'Flights with unknown carrier' : """
        SELECT COUNT(*) FROM flights f
        LEFT JOIN carriers c ON f.carrier_code = c.carrier_code
        WHERE c.carrier_code IS NULL
    """,
    'Flights with unknown origin' : """
        SELECT COUNT(*) FROM flights f
        LEFT JOIN airports a ON f.origin = a.airport_code
        WHERE a.airport_code IS NULL
    """,
    'Flights with unknown dest' : """
        SELECT COUNT(*) FROM flights f
        LEFT JOIN airports a ON f.dest = a.airport_code
        WHERE a.airport_code IS NULL
    """,
}

print('=' * 50)
print(' REFERENTIAL INTEGRITY')
print('=' * 50)

all_ok = True
with engine.connect() as conn:
    for label, query in integrity_queries.items():
        count = conn.execute(text(query)).scalar()
        flag = 'OK ' if count == 0 else 'XX '
        if count > 0:
            all_ok = False
        print(f' {flag} {label:<35} : {count:>8,}')

print()
print('All foreign key relationships intact' if all_ok else 'Integrity issues found — review above')
```

```
=====
REFERENTIAL INTEGRITY
=====
OK  Flights missing delay record      :    0
OK  Flights with unknown carrier     :    0
OK  Flights with unknown origin      :    0
OK  Flights with unknown dest        :    0

All foreign key relationships intact
```

```
# Cell 8 — Final Summary
print()
print('=' * 55)
print(' PHASE 1 DATA QUALITY — FINAL SUMMARY')
print('=' * 55)
print()
print(' Dataset : BTS On-Time Performance 2025')
print(' Database : Neon PostgreSQL (cloud)')
print(' Tables   : flights, delays, airports, carriers')
print()
print(' Checks performed:')
print(' [1] Row counts verified across all 4 tables')
print(' [2] Null check on all critical columns')
print(' [3] Duplicate flight detection')
print(' [4] Outlier detection (extreme delays, zero distance)')
print(' [5] Date range and monthly coverage verified')
print(' [6] Referential integrity across all foreign keys')
```

```
print(' [6] Referential integrity across all foreign keys ')
print()
print(' Phase 1 Data Setup : COMPLETE')
print('=' * 55)
```

=====

PHASE 1 DATA QUALITY — FINAL SUMMARY

=====

Dataset : BTS On-Time Performance 2025
Database : Neon PostgreSQL (cloud)
Tables : flights, delays, airports, carriers

- Checks performed:
- [1] Row counts verified across all 4 tables
 - [2] Null check on all critical columns
 - [3] Duplicate flight detection
 - [4] Outlier detection (extreme delays, zero distance)
 - [5] Date range and monthly coverage verified
 - [6] Referential integrity across all foreign keys

Phase 1 Data Setup : COMPLETE

=====